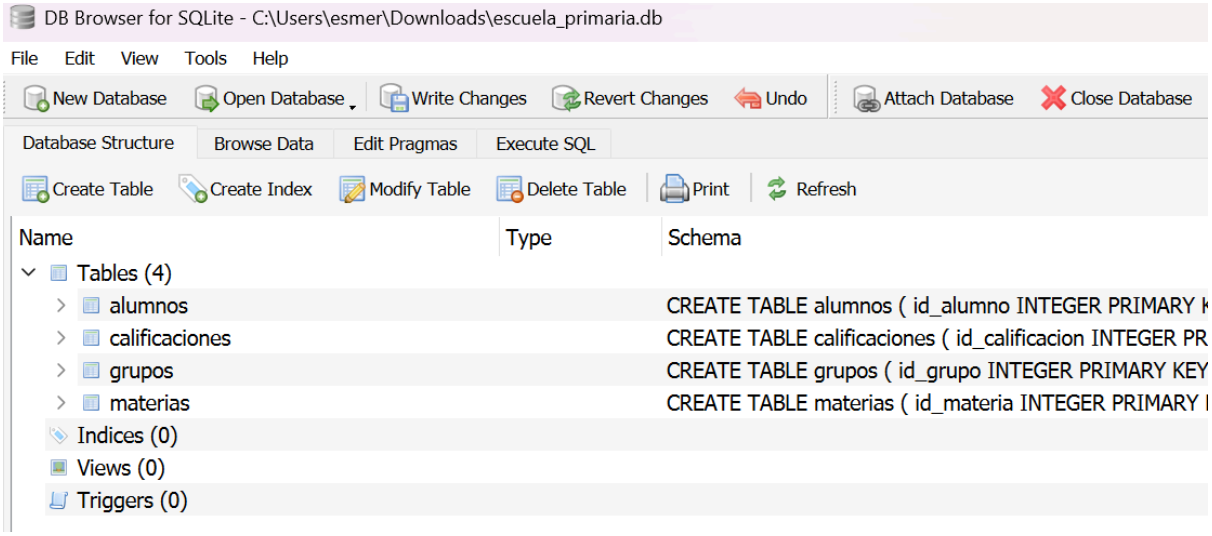
Práctica Join

- Para esta actividad creé 4 archivos relacionados con mi trabajo (actualmente soy docente de adiestramiento en artes visuales). En formato csv (calificaciones, alumnos, materias y grupos) recopilan datos sobre distintos casos relacionados al alumnado de una escuela.

- Se crea la base de batos “escuela_primaria”.



  - En la tabla alumnos: Id de alumno, nombre, apellido, edad y id de grupo.
  - En la tabla calificaciones: Id de calificación, id de alumno, id de materia y calificación.
  - En la tabla grupos: Id grupo, nombre de grupo, grado y turno.
  - En la tabla materias: Id de materias y materia.

Cada tabla desarrolla los datos de los distintos contenidos.

- Inner Join

```
query = """
SELECT alumnos.id_alumno,
       alumnos.nombre,
       alumnos.apellido,
       grupos.nombre_grupo,
       grupos.grado,
       grupos.turno
FROM alumnos
INNER JOIN grupos ON alumnos.id_grupo = grupos.id_grupo;
"""

df = pd.read_sql_query(query, conn)
df
```

	id_alumno	nombre	apellido	nombre_grupo	grado	turno
0	1	Juan	Pérez	1A	1	Matutino
1	2	María	López	1A	1	Matutino
2	3	Pedro	García	2A	2	Matutino
3	4	Ana	Hernández	2A	2	Matutino
4	5	Luis	Martínez	3B	3	Vespertino
5	6	Sofía	Ramírez	3B	3	Vespertino
6	7	Carlos	Torres	4B	4	Matutino
7	8	Daniela	Vega	4B	4	Matutino

Se juntan los datos de los alumnos junto con los datos de los grupos a los que pertenecen. Detecta los alumnos que sí tienen un grupo asignado

- Left Join

```
query = """
SELECT alumnos.id_alumno,
       alumnos.nombre,
       alumnos.apellido,
       calificaciones.calificacion
FROM alumnos
LEFT JOIN calificaciones ON alumnos.id_alumno = calificaciones.id_alumno;
"""

df = pd.read_sql_query(query, conn)
df
```

	id_alumno	nombre	apellido	calificacion
0	1	Juan	Pérez	5.5
1	1	Juan	Pérez	8.5
2	2	María	López	7.0
3	2	María	López	9.0
4	3	Pedro	García	4.0
5	3	Pedro	García	6.0
6	4	Ana	Hernández	10.0
7	5	Luis	Martínez	7.5
8	6	Sofía	Ramírez	6.5
9	7	Carlos	Torres	8.0
10	8	Daniela	Vega	5.0
11	8	Daniela	Vega	9.5

Trae las calificaciones que correspondan a cada alumno. Conserva todos los datos de los nombres de los alumnos aunque no estén calificados.

- Case when

```
query = """
SELECT alumnos.nombre,
       alumnos.apellido,
       materias.nombre_materia,
       calificaciones.calificacion,
       CASE
            WHEN calificaciones.calificacion >= 6 THEN 'Aprobado'
            ELSE 'Reprobado'
       END AS estado
FROM calificaciones
INNER JOIN alumnos ON calificaciones.id_alumno = alumnos.id_alumno
INNER JOIN materias ON calificaciones.id_materia = materias.id_materia;
"""

df = pd.read_sql_query(query, conn)
df
```

En caso de que el alumno tenga calificación menor a 6 se registra en una nueva columna como reprobado. Si es mayor a 6 se registra como aprobado.

- SubQueries

```
query = """
SELECT grupos.nombre_grupo,
       grupos.grado,
       ROUND(AVG(sub.promedio_alumno), 2) AS promedio_grupo
FROM grupos
INNER JOIN alumnos ON grupos.id_grupo = alumnos.id_grupo
INNER JOIN (
    SELECT id_alumno,
           AVG(calificacion) AS promedio_alumno
    FROM calificaciones
    GROUP BY id_alumno
) sub ON alumnos.id_alumno = sub.id_alumno
GROUP BY grupos.id_grupo;
"""

df = pd.read_sql_query(query, conn)
df
```

Permite obtener el promedio del grupo al obtener la calificación del alumno, relacionarla con el grupo y finalmente obtener el promedio por grupo.

```
query = """
SELECT *
FROM alumnos
WHERE id_alumno IN (
    SELECT id_alumno
    FROM calificaciones
);
"""

df = pd.read_sql_query(query, conn)
df
```

Identifica únicamente a los alumnos que han sido evaluados.

```
query = """
SELECT *
FROM alumnos
WHERE id_alumno NOT IN (
    SELECT id_alumno
    FROM calificaciones
);
"""

df = pd.read_sql_query(query, conn)
df
```

Detecta alumnos que aún no han sido evaluados.